

Compilation, early-exit rollouts and wind estimation

Round 3 on the multithreaded branch: making every in-flight computation predictable, and finding what actually crashes the vehicle

2026-09-27

Table of contents

1	Summary	1
2	Making JIT compilation impossible in flight	2
2.1	Why “warm it up in <code>__init__</code> ” wasn’t enough	2
2.2	Ahead-of-time compilation	2
3	Early-exit rollouts	3
3.1	When to start the next rollout	3
4	Wind estimation	4
5	Controller robustness	4
6	Ablation	5
6.1	What was crashing the vehicle	5
6.2	Still open	7
7	Should the fast loop move to C++?	7

1 Summary

	round 2	round 3
in-flight JIT compilations	possible (signature-dependent)	impossible (AOT executables)
start-up compilation	~20 s	~ 6 s (persistent cache)
control tick (compute)	93 μ s, ~6 XLA dispatches	31 μs, 1 dispatch
one rollout	60–190 ms (full 30 s horizon)	3–4 ms (early exit, same plan)
ticks flown on an expired plan	58–74 %	0 % (pre-emptive replanning)

	round 2	round 3
NR pitch/yaw-rate commands in crashes	up to 260 000 rad/s	clipped to ± 0.8 rad/s
attitude upsets (SITL, events + process)	8 of 18 flights	0 of 10 (NR reference from plan)

Every change is behind a command-line flag, so each can be switched off to reproduce earlier behaviour. Section 6 measures which control-side changes actually reduce crashes.

2 Making JIT compilation impossible in flight

2.1 Why “warm it up in `__init__`” wasn’t enough

`jax.jit` doesn’t compile a *function*; it compiles one executable per **call signature**. The cache key includes each argument’s

- shape and dtype;
- **weak type**: a Python float is a *weakly typed* scalar, `np.float64(0.8)` or a float64 array is not;
- **committed-ness**: whether an array already lives on a specific device (the output of a jitted function does; a NumPy array doesn’t);
- static arguments, by hash.

The original warm-up called `NR_tracker_original` with arrays built at start-up. In flight, from the second RTA tick on, `last_input` was the controller’s *own output*, a committed device array. That is a different signature, so JAX traced and compiled again: 322 ms in the air, followed by more compiles for `dynamic_slice`, `concatenate` and the wind model. Adding more warm-up calls with “the right” arguments works only until the next refactor changes a type.

2.2 Ahead-of-time compilation

JAX can compile for an explicit signature *without calling the function*:

```
f64 = lambda *shape: jax.ShapeDtypeStruct(shape, jnp.float64)
compiled = jax.jit(control_step).lower(f64(9), f64(5), f64(4),
    ↪ ...).compile()
```

`compiled` is a `jax.stages.Compiled`: one executable, for exactly that signature. **It never recompiles.** Called with anything else, it raises `TypeError: Argument types differ from the types for which this computation was compiled`, which fails loudly on the bench instead of silently stalling a flight. Every call site canonicalises its inputs first:

```
def as_f64(x, shape):
    return np.ascontiguousarray(x, dtype=np.float64).reshape(shape)
```

and the controller’s state (`last_input`) is kept as a float64 NumPy array. Four executables are built at start-up (`px4_rta_mm_gpr/control_kernels.py` and `RolloutEngine`):

executable	inputs	replaces
<code>control_step</code>	NR state (9), planar state (5), last input (4), NR reference (4), plan row (5), feedforward row (2), K (2×5), clip bounds	<code>NR_tracker_original</code> , slicing, <code>u_applied</code> , <code>hstack</code>
<code>wind_model</code>	NR state, input	<code>OBS_DYN @ dynamics(...)</code>
<code>linearize</code>	planar state, input	<code>jitted_linearize_system</code>
<code>rollout</code>	t_0 , state, x_{pert} , K_{fb} , K_{ref} , GP data $\times 2$, goal	<code>jitted_rollout</code> + <code>collection_id_jax</code>

Fusion. A control tick used to launch about six separate XLA computations, each with its own Python dispatch overhead. `control_step` is one executable: **31 μs instead of 93 μs** per tick, with results identical to the unfused functions (maximum difference 0 over 50 random inputs).

Persistent cache. `jax_setup.py` points `jax_compilation_cache_dir` at `~/.cache/px4_rta_mm_gpr/jax`. Compiled executables are written to disk and reused by later runs *and* by the rollout worker process, so start-up compilation went from about 20 s to about 6 s. Set `PX4_RTA_JAX_CACHE=''` to disable it.

3 Early-exit rollouts

A rollout integrates the embedding system and the reference for the whole horizon (30 s = 3000 steps), then `collection_id_jax` finds the **first** row where the tube leaves the threshold. That row is typically 40–50 steps in. Every row after it is computed and never used; the over-approximation has usually diverged by then anyway (to 10^{195} within 3 s).

`rollout_until_violation` (`jax_mm_rta/mm_rta.py`) replaces the `lax.scan` with a `lax.while_loop` that checks each new row with the same test and stops **margin** steps after the first violation. Both versions call the *same* step function (`_make_step`), and the scan is causal, so:

- every computed row is **bit-identical** to the full rollout (`np.array_equal` on six plans taken from a logged flight);
- the violation index is identical;
- the cost is proportional to (violation + margin), not the horizon: **3–4 ms instead of 62–108 ms** on this machine, and the horizon (`--tube-horizon`) becomes just an upper bound.

Plans, IPC messages and log files keep only the valid rows (about 90 instead of 3001).

3.1 When to start the next rollout

A plan is certified up to its violation time, `collection_time`. The original code started the next rollout *at* that time, which is too late by construction: the new plan arrives one rollout latency after the old certification ended, and in the meantime the controller flies an uncertified reference.

The rule now is that **the next plan must be installed before the current one expires**. The rollout timer starts a rollout at `collection_time - lead`, where `lead` is twice the worst rollout latency over the last 50 rollouts, and at least two control periods (the timer’s own granularity). Each plan still keeps a 1.0 s margin past its violation (`--tube-margin`) as a fallback if a rollout is ever late, and every control tick on an expired plan is counted as a *certification gap* in the timing summary and the log (`plan_expired`). In SITL: 0 of 2000 ticks.

4 Wind estimation

Only learn when the model is right. Before $t = 15$ s, PX4’s position controller flies the vehicle, but the EKF’s model used `last_input`, a constant placeholder `[m g, 0.01, 0.01, 0.01]`. The “wind” it estimated during the climb was model error, and (now that the GP buffers actually update) it was fed into the GP. The EKF now runs only while the node’s own body-rate commands fly the vehicle, is reset when that starts, and the GP learns only then.

Use all the data. The EKF previously used one odometry sample per 100 ms. It now runs in the odometry callback at 40 Hz (the relay’s rate) with the random-walk process noise scaled by the actual sample interval, so the filter’s time constants are unchanged while it sees four times as many measurements. The model prediction is the AOT-compiled `wind_model`.

Which variable each GP depends on. By design, the y-wind is learned as a function of altitude (z) and the z-wind as a function of lateral position (y). The training data followed that, but the rollout queried *both* GPs at p_z . The z-wind GP is now queried at p_y (`WZ_INPUT_IDX`).

5 Controller robustness

NR anti-windup. The NR tracker integrates its own input ($u \leftarrow u + \alpha \dot{u} \Delta t$), and an integral CBF keeps the rates within ± 0.8 rad/s. In every flight that did *not* crash, the pitch and yaw rates stayed at or inside ± 0.80 rad/s. In every crash they diverged *before* ground contact, to 84, 62 141 and 260 889 rad/s. PX4 clamps such commands to its maximum body rates, so the vehicle spins at full rate. The stored input is now clipped to the CBF limits (a no-op in normal flight). `--no-nr-anti-windup` disables it.

Yaw unwrapping. `adjust_yaw` counted $\pm 2\pi$ only when consecutive samples straddled $\pm 0.9\pi$. A jump across the wrap by less than that band (e.g. $+170^\circ \rightarrow -100^\circ$ during an upset) produced a spurious 2π error. It now accumulates the wrapped increment between samples (like `numpy.unwrap`), which is identical in normal flight.

RTA thrust limits and vehicle mass. The RTA input limits [13, 21] N were tuned for the 1.75 kg vehicle (hover 17.2 N, braking margin +3.8 N). The SITL model is 2.0 kg (hover 19.6 N), which leaves only **+1.4 N (0.7 m/s²)** to stop a descent. In the flights that hit the ground, thrust was pinned at 21 N for 58 % of the descent, and the rollout’s own reference descended at 4 m/s through $z = 0$. `--thrust-limits-mass-scaled` keeps the hardware *ratios* ($0.757\text{--}1.224 \times$ hover thrust): exactly [13, 21] N on hardware, [14.9, 24.0] N in SITL.

NR reference from the plan. See Section 6; this is the change that stopped the upsets.

Softer RTA entry. `--entry-ramp` (off by default, see Section 6) moves the rollout goal from the entry position toward `GOAL_STATE` at bounded speed (`--ramp-speed-y`, `--ramp-speed-z`) instead of jumping from $(3.9, -12.5)$ to $(0, -0.6)$ at once.

Note that the certificate compares the tube to the *reference*; neither knows where the ground is. A reference that descends through $z = 0$ can be perfectly “certified”.

6 Ablation

All flights use the round-3 infrastructure (AOT kernels, early-exit rollouts, pre-emptive replanning, NR anti-windup, wind EKF gating, `--executor events --rollout-backend process`); only the control-side options change. **Every configuration held 100 Hz with a p99 period of 11–13 ms and 0 % certification gaps**, so these differences are not about timing.

A flight is an **upset** if roll or pitch exceeds 60° or the vehicle flies more than 8 m sideways during the RTA phase, and a **touch** if it reached $z > 0$ in the node’s frame. In sim the node adds 0.5 m to z near the ground, so a touch means coming within about 0.5 m of the ground, not necessarily a crash.

configuration	flights	upsets	touches	median max pitch
infrastructure only	4 + 5	2	3	25–53°
+ mass-scaled thrust limits	4	3	0	55°
+ entry ramp	4	3	1	84°
+ thrust limits + entry ramp	4	2	0	67°
infrastructure, no GP learning	4	1	2	27°
+ NR reference from plan	5	0	0	2.4°
+ NR reference from plan + thrust limits	5	0	0	2.7°

Full per-flight numbers are in `docs/data/round3_flights.csv`.

6.1 What was crashing the vehicle

The RTA flies **thrust and roll** along its plan; the NR tracker keeps **pitch and yaw**. But the NR tracker still computed its Newton step for all four inputs against its *own* reference: $y = 0$ and $z = -12.5 + 0.1t$. At RTA entry the vehicle is at $y = 3.9$ m, so the NR tracker saw a ~ 4 m lateral error. The coupled inverse Jacobian answered it partly through the only channels it still controlled,

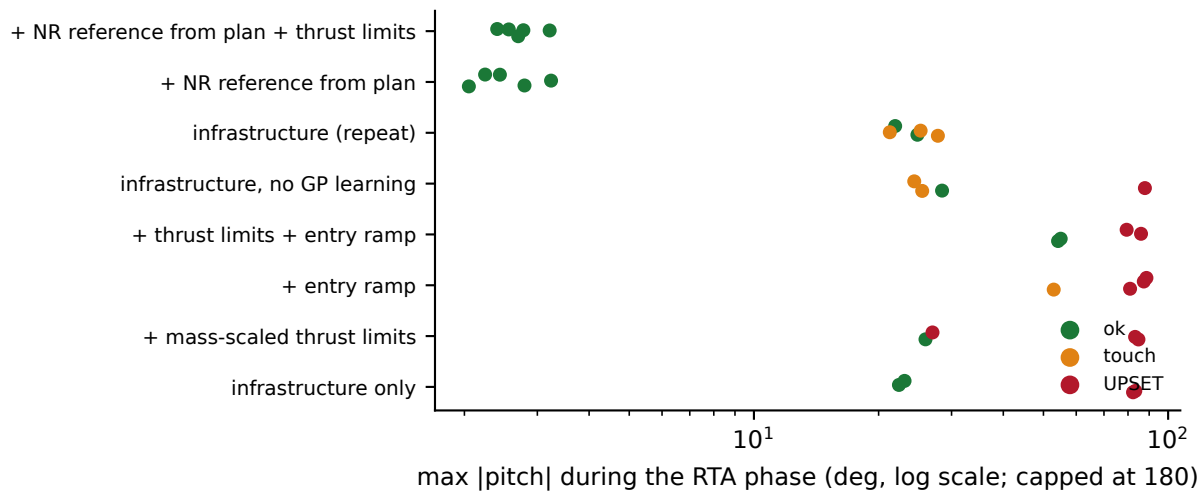


Figure 1: Maximum pitch per flight (log scale). Only the NR-reference change moves the distribution.

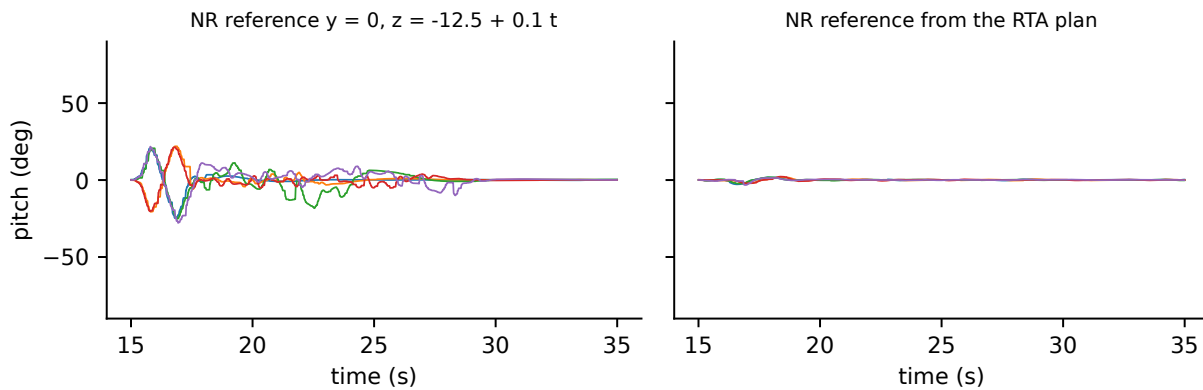


Figure 2: Pitch during the RTA phase, five flights each. Left: the NR tracker's own reference; right: the NR reference taken from the RTA plan.

pitch and yaw. Pitch swung between about -37° and $+52^\circ$, the thrust vector tilted out of the y - z plane that the RTA’s planar model assumes, and in about half of the flights the vehicle upset.

`--nr-ref-from-plan` (default on) gives the NR tracker the plan’s y, z at its own prediction horizon ($t + T_{\text{lookahead}} = 0.8$ s), so the two controllers agree and the NR tracker only has x and yaw left to correct. With it, **10 of 10 flights completed with no upset and no ground contact**, with maximum pitch around 2 – 3° and attitude within about 32° . Against 8 upsets and 6 touches in the 18 flights without it, this is not noise.

Two other observations:

- The **entry ramp** made things worse (12 m/s descents in some flights): chasing a moving goal kept the reference descending. It is now off by default.
- The **mass-scaled thrust limits** had no measurable effect on outcomes in this manoeuvre. They stay on, because they give the SITL vehicle the braking authority the limits were tuned for (and are exactly $[13, 21]$ N on hardware).

6.2 Still open

- With the NR fix, the vehicle settles around 3 m instead of at `GOAL_STATE` (0.6 m). Earlier flights settled at the same altitude after diving to the ground and back. The GP learns a steady ~ 1 N “vertical wind” (`wz`), which is really the throttle-to-force map disagreeing with the model by about 5 %. The reference LQR has no integral action, so a constant disturbance leaves a steady-state offset. Correcting the thrust map, or adding integral action to the reference, would address it.
- The certificate compares the tube with the *reference* only; nothing checks the ground.

7 Should the fast loop move to C++?

Not for this problem. Every configuration in the ablation already held 100 Hz with an 11–13 ms p99 period, 0 % certification gaps and a 31 μ s control computation, and the upsets came from the control *logic*. A C++ loop would have flown the same conflicting references.

For hardware, yes. It is the natural next step. A small C++ node would own the heartbeat, arming, mode logic, and the 100 Hz law (the RTA feedback $u = u_{\text{ff}} - K(x - x_{\text{ref}})$ clipped, plus the NR tracker with a finite-difference Jacobian). The Python node would publish certified plans (`tube`, `reference`, `feedforward`, `collection_time`) as messages. Benefits:

1. **Determinism on a slower onboard computer.** A Jetson or Raspberry Pi has far less margin than this 14-core desktop.
2. **A watchdog that doesn’t depend on what it watches.** If the planner stalls, crashes or stops delivering certified plans, the C++ node detects plan expiry and switches to a backup (PX4 hold or land). This is what runtime assurance promises.
3. **No GIL or GC anywhere near the control loop**, instead of mitigating them.